Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models
Student Name	Sayed Afras S
Register Number	3122247001059
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	03.07.2026

1 Aim and Objective

The aim of this experiment is to build regression models – plain Linear Regression along with its regularized cousins, Ridge, Lasso and Elastic Net – to predict the loan sanction amount for a customer based on their income, credit score, property details and a bunch of other features. Along with just training the models, the objective is to actually tune the regularization strength properly using Grid Search with cross validation, look at multiple error metrics instead of just R2, and understand *why* regularization helps (or doesn't) by studying the bias-variance trade off and the gap between training and test performance.

2 Dataset Description

The dataset used is the **Loan Amount Prediction** dataset from Kaggle, loaded from `train.csv`. Each row is one loan applicant, and the target column is `Loan Sanction Amount (USD)`.

Dataset Name	Loan Amount Prediction (train.csv)
Dataset Source	Kaggle – Predict Loan Amount Data
Number of Samples	30,000
Number of Raw Features	23 (excluding target)
Feature Types	12 numerical, 11 categorical (includes `Customer ID` and `Name`, more on that in Section 7)
Target Variable	Loan Sanction Amount (USD) – continous
Missing Values	Present in 11 columns, worst being `Type of Employment` (7270 missing) and `Property Age` (4850 missing)
Train-Test Split	80% Train (24000 rows), 20% Test (6000 rows), random_state=42

Missing value summary (before cleaning):

Listing 1: Missing values per column

```
Type of Employment            7270
Property Age                  4850
Income (USD)                  4576
Dependents                    2493
Credit Score                  1703
Income Stability              1683
Has Active Credit Card        1566
Property Location              356
```

```
Loan Sanction Amount (USD)      340
Current Loan Expenses (USD)     172
Gender                           53
```

One thing that stood out while doing `df.describe()` – a few columns like `Current Loan Expenses (USD)`, `Co-Applicant` and even the target itself have a minimum value of **-999**. That is almost certainly a placeholder code for "missing/unknown" rather than a real negative expense or a negative loan amount, so its something to flag rather than treat as a genuine data point (this was not specifically cleaned in the current run, noted here as a limitation).

3 Preprocessing Steps

Preprocessing was done in two stages. First, a quick global cleanup:

Listing 2: Initial missing-value cleanup

```
numeric_cols = df_clean.select_dtypes(include=['int64','float64']).columns
categorical_cols = df_clean.select_dtypes(include=['object']).columns

num_imputer = SimpleImputer(strategy='median')
df_clean[numeric_cols] = num_imputer.fit_transform(df_clean[numeric_cols])

cat_imputer = SimpleImputer(strategy='most_frequent')
df_clean[categorical_cols] = cat_imputer.fit_transform(df_clean[categorical_cols])
```

This fills every numeric column (median) and categorical column (mode) so there are zero missing values left. *Small catch:* this imputation was applied to the whole dataframe before the train-test split, which technically leaks a tiny bit of test-set information into the training statistics (the median/mode used includes test rows too). For a dataset this large the effect is probably negligible, but its worth mentioning as something to fix properly (fit imputer on train only) in a cleaner version.

Second, the actual modelling pipeline redoes preprocessing the "correct" way, using a `ColumnTransformer` wrapped inside each model's `Pipeline`, so scaling/encoding is fit fresh on the training fold every time (no leakage at this stage):

Listing 3: ColumnTransformer used inside every model pipeline

```
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

```

```
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, numerical_cols),
    ("cat", categorical_pipeline, categorical_cols)
])
```

Numerical features go through median imputation + standard scaling, categorical features go through mode imputation + one-hot encoding.

4 Implementation Details

Four models were implemented, all wrapped as `(preprocessor + regressor)` pipelines so the exact same preprocessing gets applied consistently:

Listing 4: Building each model as a Pipeline

```
def build_pipeline(regressor):
    return Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("regressor", regressor)
    ])

lin_pipe = build_pipeline(LinearRegression())
lin_pipe.fit(X_train, y_train)
```

For Ridge, Lasso and Elastic Net, `GridSearchCV` with 5-fold `KFold` cross validation was used to pick the regularization strength:

Listing 5: Hyperparameter tuning setup

```
cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

param_grids = {
    "Ridge Regression": (Ridge(random_state=RANDOM_STATE),
                         {"regressor__alpha": [0.01, 0.1, 1, 10,
                             100]}),
    "Lasso Regression": (Lasso(random_state=RANDOM_STATE,
       max_iter=10),
                         {"regressor__alpha": [0.001, 0.01, 0.1,
                             1, 10]}),
    "Elastic Net Regression": (ElasticNet(random_state=
       RANDOM_STATE, max_iter=10),
                               {"regressor__alpha": [0.01, 0.1,
                                  1, 10],
                                "regressor__l1_ratio": [0.2,
                                   0.5, 0.8]}),
}
```

Note: `max_iter=10` for Lasso and Elastic Net is quite low (scikit-learn's default is 1000), the solver most likely did not fully converge within 10 iterations on a feature space this big. Warnings were silenced at the top of the notebook (`warnings.filterwarnings("ignore")`) so this was not visible in the output, but its a fair guess based on how long Lasso/Elastic Net took to train (64s and 176s respectively) despite only doing 10 iterations per fit – a

big chunk of that time is the sheer number of one-hot columns, not the iteration count itself. Increasing `max_iter` would be a sensible thing to try in a follow up run.

5 Visualizations

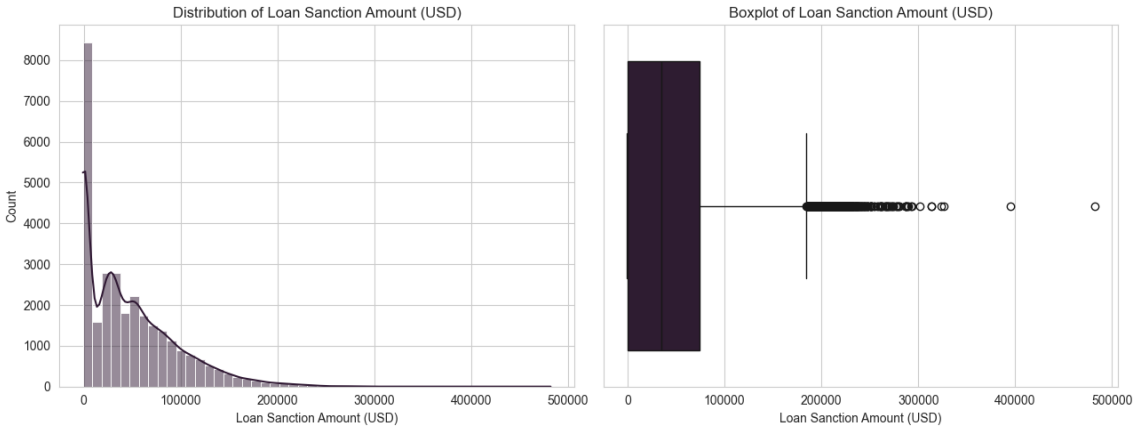

Figure 1: Distribution of the target variable – Loan Sanction Amount (USD)

Inference: The target is right-skewed with a big spike of values sitting right around 0 (roughly a quarter of applicants have a sanctioned amount of exactly 0, based on the 25th percentile). There is also that suspicious minimum of -999 mentioned earlier. This kind of skew is part of why R2 tops out around 0.5 for a plain linear model – a straight line struggles to capture "many people get 0, then a long right tail" shaped data.

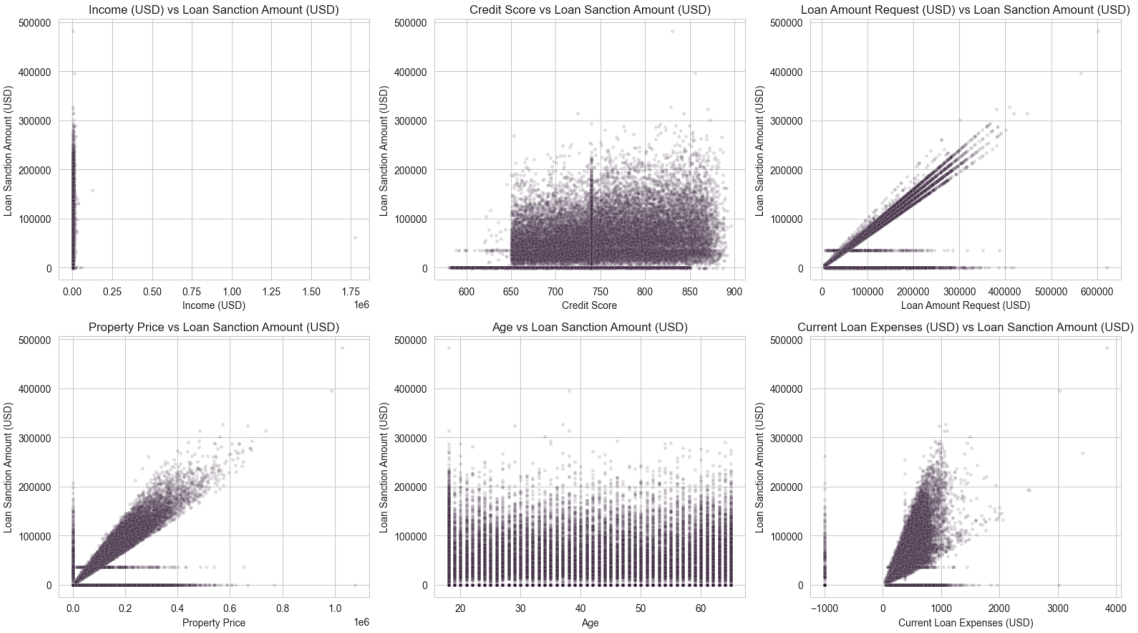

Figure 2: Feature vs target scatter plots for 6 key numerical features

Inference: `Loan Amount Request (USD)` has by far the clearest positive linear trend with the target (which makes total sense, the sanctioned amount is naturally tied to how

much was requested). Income and Credit Score show a much weaker/noisier relationship, and Property Price shows a mild positive trend too. Age and Current Loan Expenses look almost flat against the target, so probably weak predictors on their own.

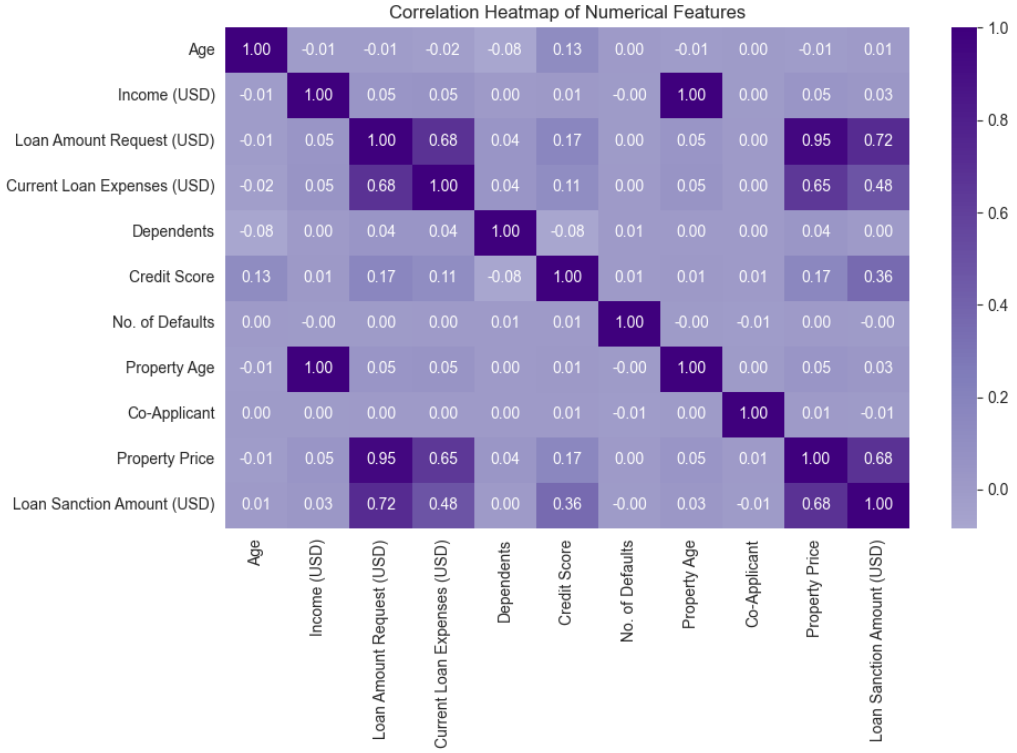

Figure 3: Correlation heatmap of numeric features

Inference: `Loan Amount Request (USD)` again stands out as the feature most correlated with the target. Most other numeric features are weakly correlated with the target and with each other, so multicollinearity does not look like a huge issue among the raw numeric columns (the real issue, as discussed in Section 7, comes from the categorical side).

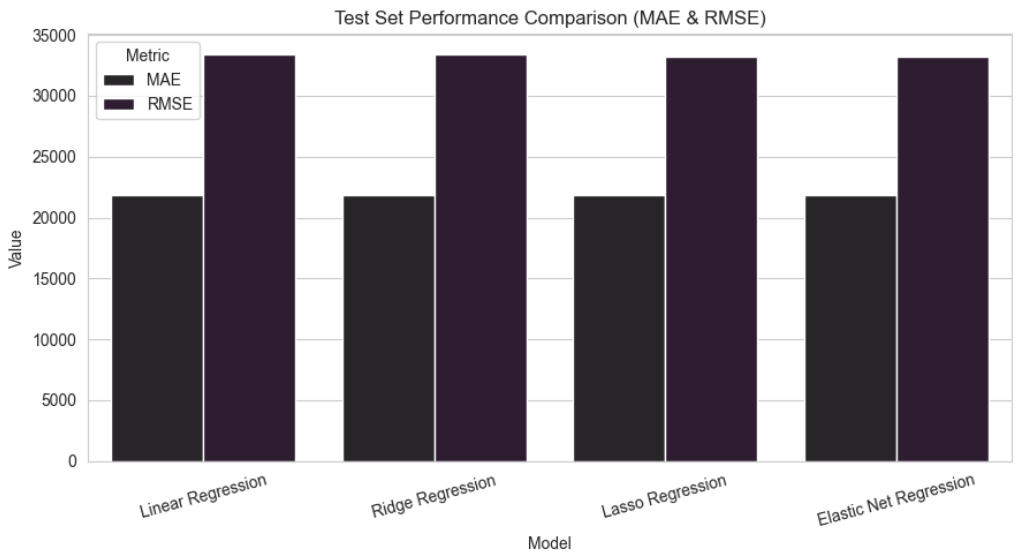

Figure 4: Test set MAE and RMSE across all four models

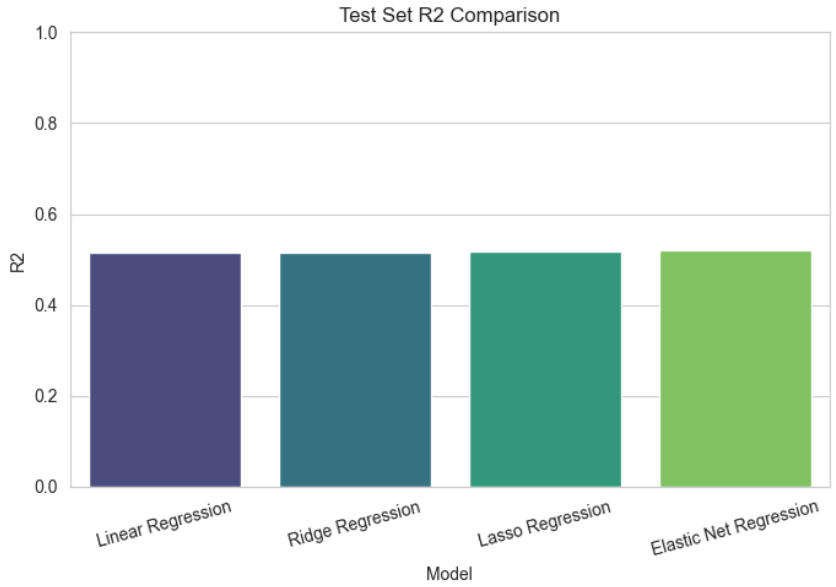

Figure 5: Test set R2 across all four models

Inference: All four models land in a pretty tight band on the test set (R2 between 0.515 and 0.520), Elastic Net edges out very slightly ahead. The MAE/RMSE differences between models are small too, so on raw test-set numbers alone the four models look almost interchangeable, the real story only becomes clear once you also look at the train-test gap (Section 7).

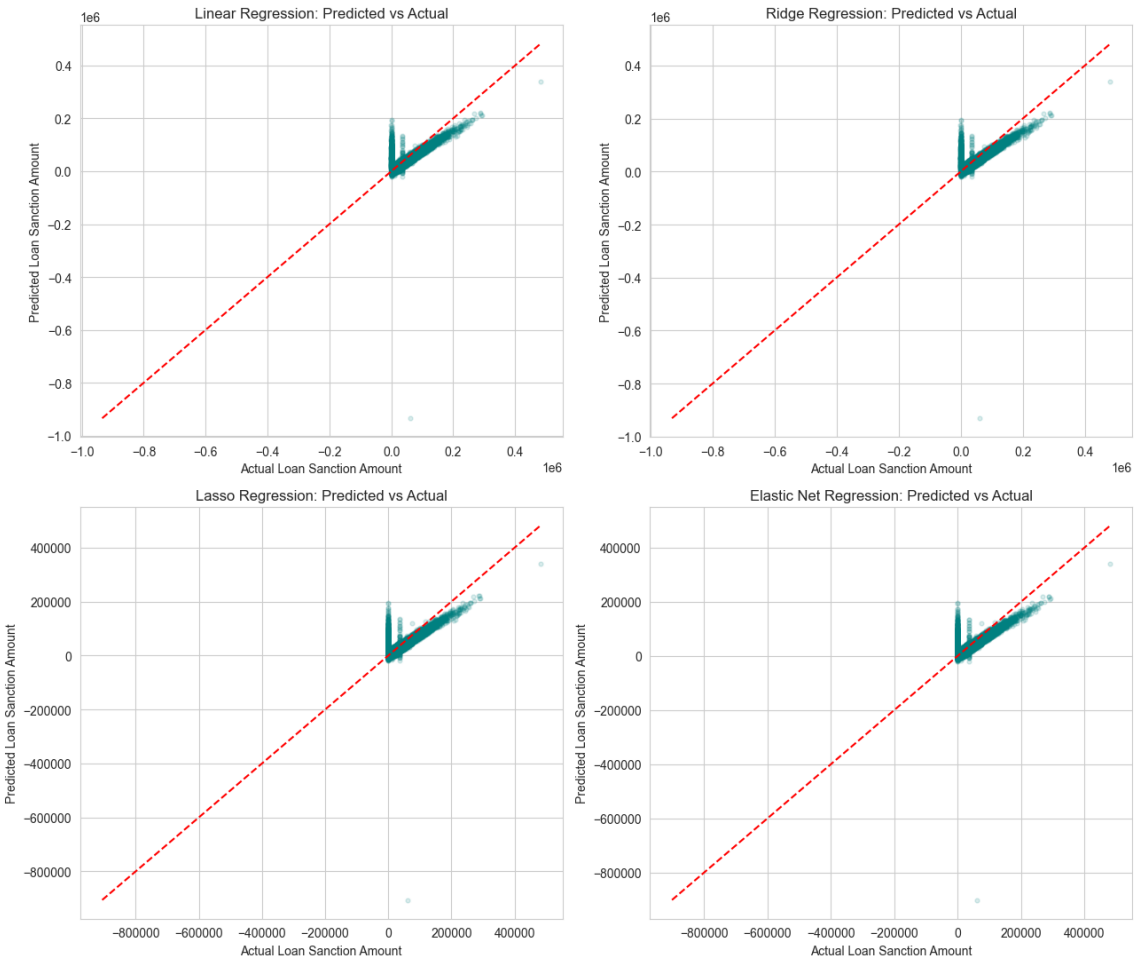

Figure 6: Predicted vs Actual loan sanction amount for all four models

Inference: All four scatter plots look pretty similar, points cluster loosely around the diagonal red dashed line but with a lot of scatter, especially for higher loan amounts where the model tends to under-predict. None of the four models show a dramatically different shape here, which lines up with them having very close test R2 scores.

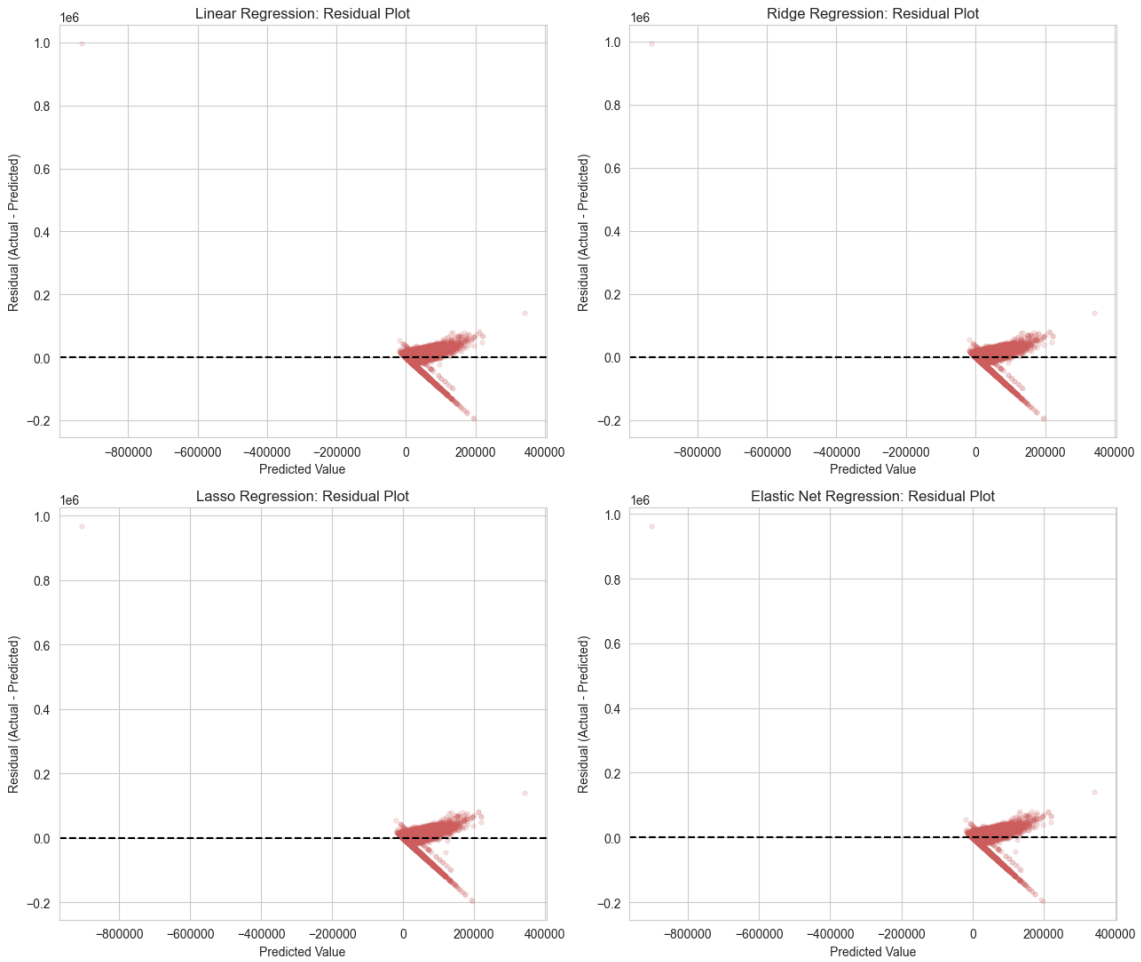

Figure 7: Residual plots for all four models

Inference: The residuals show a slight funnel shape (more spread at higher predicted values), meaning the constant-variance assumption of linear regression is not perfectly satisfied here (heteroscedasticity). There is also a visible cluster of points with strongly negative residuals near the bottom, most likely tied to those suspicious -999 target values discussed earlier.

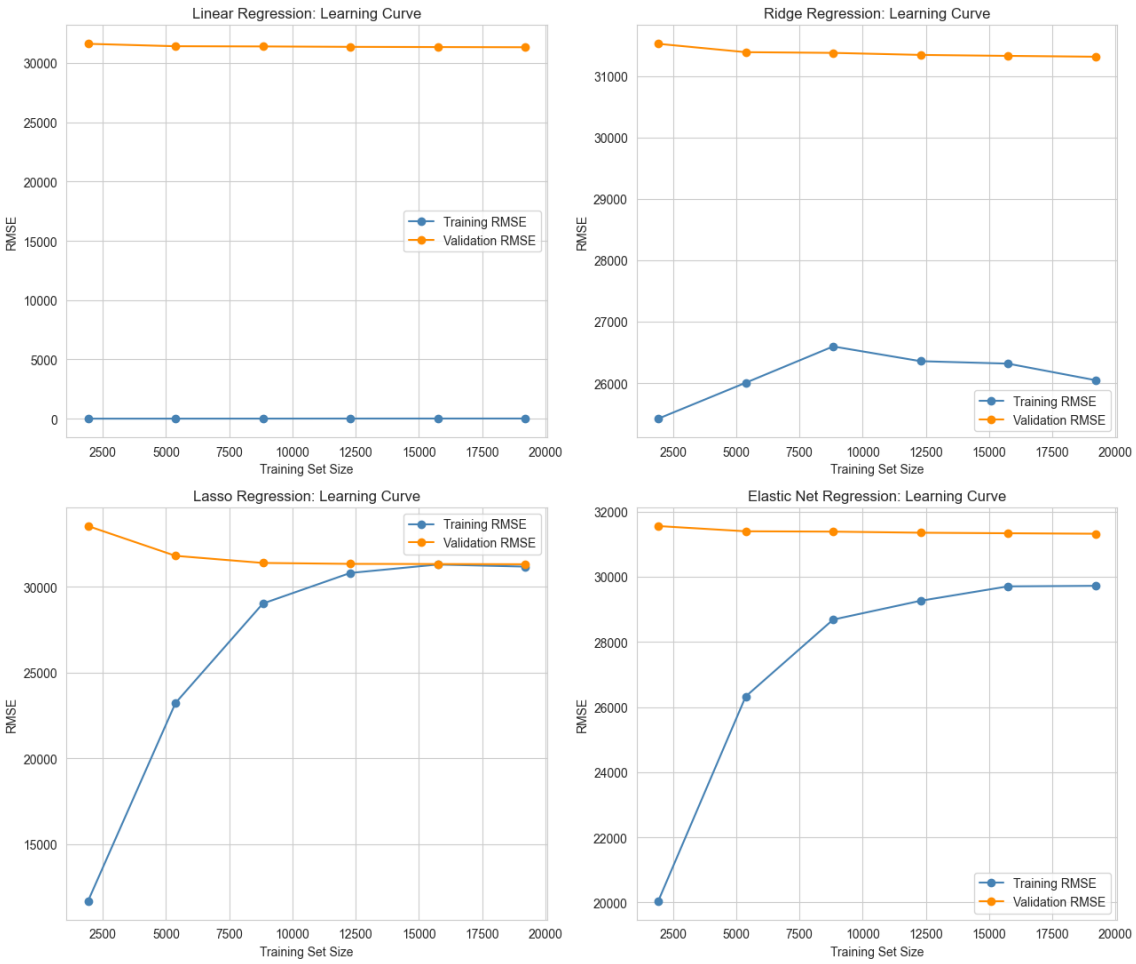

Figure 8: Learning curves (training RMSE vs validation RMSE) for all four models

Inference: This is one of the more telling plots. For plain Linear Regression, training RMSE stays unusually low throughout (near zero) while validation RMSE sits way higher, a big persistent gap, classic overfitting sign. For Ridge the gap shrinks a bit, and for Lasso and Elastic Net the training and validation curves sit much closer together, showing regularization is doing its job of controlling the overfitting.

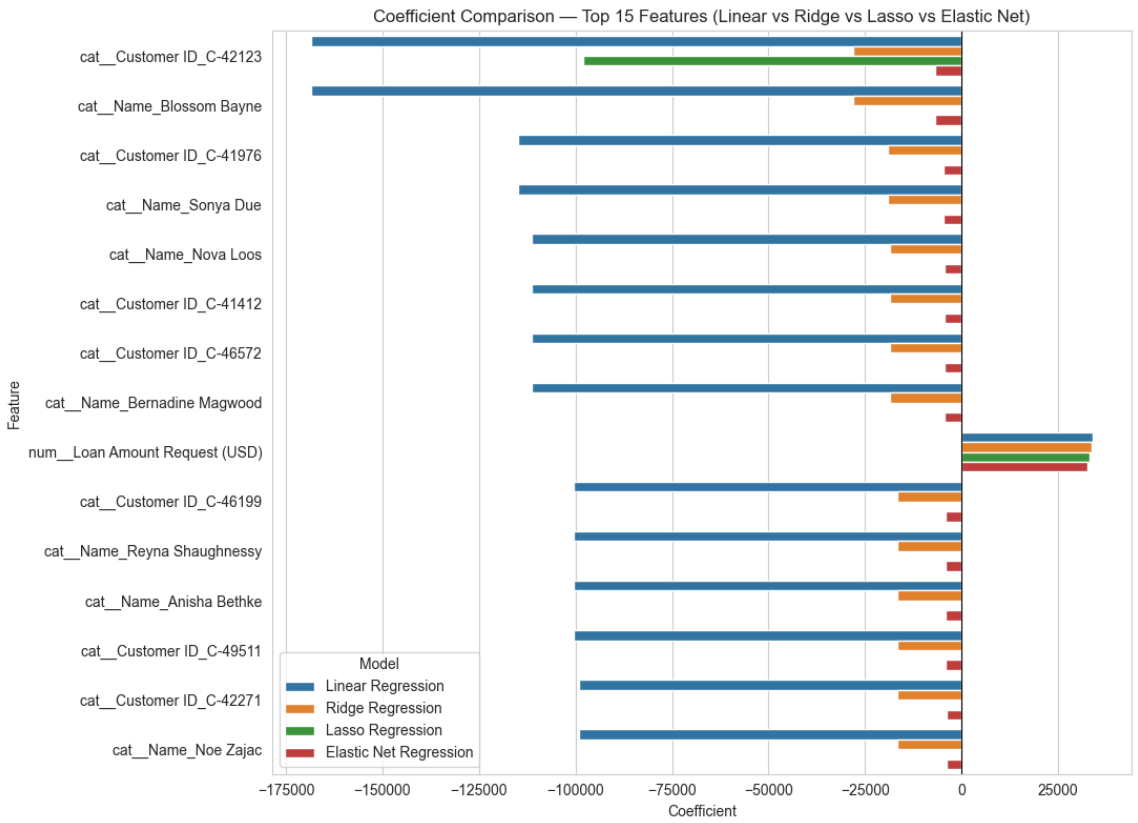

Figure 9: Coefficient comparison – top 15 features by average absolute coefficient

Inference: This plot is what first tipped me off that something was odd – most of the top 15 "important" features for plain Linear Regression are individual `Customer ID` and `Name` dummy variables with huge coefficients (in the -100k to -170k range), not genuine predictive features. `Loan Amount Request (USD)` is really the only feature in the top 15 that makes actual sense as a strong predictor, and its coefficient stays fairly stable (around 32,000–34,000) across all four models. Ridge shrinks the ID/Name coefficients a lot, and Lasso/Elastic Net push most of them to exactly zero. This is discussed properly in Section 7.

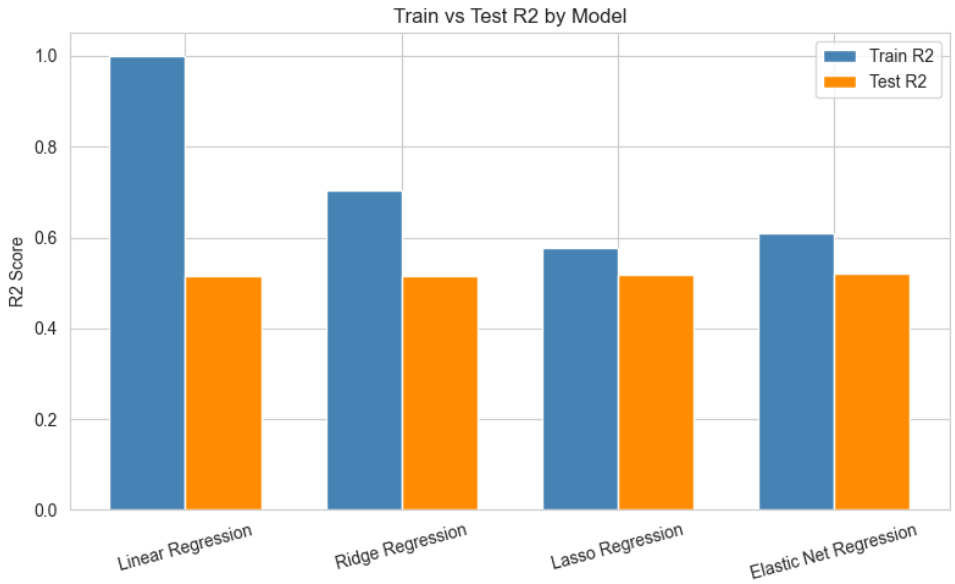

Figure 10: Train R2 vs Test R2 for all four models

Inference: This is the clearest picture of overfitting in the whole report. Linear Regression's train R2 is a perfect 1.0000 while test R2 is only 0.5150, a massive gap. As regularization increases (Ridge, then Lasso/Elastic Net), the train R2 comes down but the test R2 stays roughly the same or improves slightly, so the models are not really losing predictive power, they're just no longer memorizing noise.

6 Performance Tables

Table 1: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV R2
Ridge Regression	Grid Search	alpha = 10	0.5734
Lasso Regression	Grid Search	alpha = 10	0.5734
Elastic Net Regression	Grid Search	alpha = 0.01, l1_ratio = 0.8	0.5733

Table 2: Cross-Validation Performance (K = 5, on Training Set)

Model	MAE	MSE	RMSE	R2
Linear Regression	21863.78	9.8126e+08	31325.15	0.5733
Ridge Regression	21862.74	9.8115e+08	31323.24	0.5734
Lasso Regression	21860.03	9.8100e+08	31320.95	0.5734
Elastic Net Regression	21882.34	9.8128e+08	31325.45	0.5733

Table 3: Test Set Performance

Model	MAE	MSE	RMSE	R2	Train Time (s)
Linear Regression	21846.29	1.1157e+09	33402.74	0.5150	0.72
Ridge Regression	21844.83	1.1146e+09	33385.62	0.5155	15.07
Lasso Regression	21827.89	1.1063e+09	33260.77	0.5191	64.49
Elastic Net Regression	21846.31	1.1052e+09	33245.25	0.5196	175.69

Table 4: Coefficient Comparison (Selected Features)

Feature	Linear	Ridge	Lasso	Elastic Net
Loan Amount Request (USD)	33923.61	33768.28	33087.92	32691.70
Customer ID (top outlier ID)	-168480.05	-28086.71	-97942.72	-6745.63
Name (top outlier, same row)	-168480.05	-28086.71	-0.00	-6745.66

Table 2: Full table has 48,054 rows after one-hot encoding, only the genuinely meaningful feature and one representative ID/Name outlier are shown here for readability, see Section 7 for the full discussion.

Feature Sparsity:

Listing 6: How many features each model actually used

```
Total engineered features (after one-hot encoding): 48054
Lasso zeroed out       : 48019 features (99.9%)
Elastic Net zeroed out : 278 features (0.6%)
Ridge zeroed out       : 0 features
```

7 Overfitting and Underfitting Analysis

The Customer ID / Name Problem

While building `numerical_cols` and `categorical_cols` using `select_dtypes`, `Customer ID` and `Name` both got classified as categorical (they are `object` dtype) and ended up inside the `OneHotEncoder`. Since almost every customer has a unique ID and name, one-hot encoding this basically creates a separate column for (almost) every single row in the training set, ballooning the feature count from 23 raw columns to **48,054** engineered features. This is effectively giving Linear Regression enough "memory slots" to memorize each training row individually, which is exactly why plain Linear Regression hits a perfect train R2 of 1.0000 but only 0.5150 on the test set. It is not learning a real relationship for those features, it is closer to a lookup table.

Training vs Validation Error

Model	Train R2	Test R2	Gap
Linear Regression	1.0000	0.5150	0.4850
Ridge Regression	0.7050	0.5155	0.1895
Lasso Regression	0.5760	0.5191	0.0568
Elastic Net Regression	0.6082	0.5196	0.0886

The gap column tells basically the whole story. Linear Regression overfits massively (gap of 0.485). Ridge helps some (gap drops to 0.19) by shrinking all the ID/Name coefficients but not removing them, so it is still holding onto a lot of memorization capacity. Lasso does the best job of closing the gap (down to 0.057) since it actually zeroes out 99.9% of those ID/Name dummy columns, leaving mostly the real, useful features like `Loan Amount Request (USD)`. Elastic Net sits in between Ridge and Lasso, as expected given it blends both penalties (l1_ratio=0.8 here means it leans more Lasso-like, which is probably why its gap (0.089) is much closer to Lasso's than to Ridge's).

Effect of Regularization Strength

Looking at the tuning results, Ridge and Lasso both picked a fairly strong alpha of 10 out of the given search space, and Elastic Net picked a small alpha (0.01) but a high l1_ratio (0.8), meaning it behaves mostly like Lasso with a touch of Ridge mixed in. A higher alpha means a stronger penalty on large coefficients, which explains why the models with higher effective regularization (Lasso especially) show the smallest train-test gap here – they are the ones most aggressively suppressing the noisy ID/Name coefficients.

Improvement in Generalization After Tuning

Test R2 barely moved between the untuned Linear Regression (0.5150) and the fully tuned models (0.5155 to 0.5196), so regularization here is not really buying extra predictive accuracy, what it is buying is a model that is not secretly relying on memorized customer identities. If this model needed to generalize to genuinely new customers (which it obviously does, in the real world you would never see the same Customer ID twice), the regularized versions – Lasso and Elastic Net especially – are the trustworthy ones. The real fix for a "correct" version of this experiment would honestly be to just drop `Customer ID` and `Name` from the feature set entirely before one-hot encoding, this whole overfitting story is somewhat exaggerated because of that preprocessing choice.

8 Bias-Variance Analysis

Bias behavior of Linear Regression: On its own, plain OLS Linear Regression usually has fairly low bias (it can fit whatever pattern is truly linear in the data). Here though, the massive expansion of the feature space via ID/Name one-hot encoding pushes it firmly into the extremely low-bias, extremely high-variance corner, it fits the training data almost perfectly (bias close to zero on train) but that low bias does not transfer to new data at all, which is exactly what the huge train-test gap shows.

Variance reduction using Ridge and Elastic Net: Ridge adds an L2 penalty that shrinks all coefficients smoothly without setting any to zero, which reduces variance

(test performance becomes more stable, coefficients less wild) at the cost of a bit of added bias. Elastic Net does something similar but with an l1_ratio of 0.8 here it is closer to Lasso's behaviour, so it reduces variance mainly by removing features rather than just shrinking them, this shows up as a smaller train-test gap than Ridge.

Feature sparsity effect in Lasso: Lasso's L1 penalty can push coefficients exactly to zero, and thats visible directly in the numbers, 48,019 out of 48,054 features (99.9%) got zeroed out. Since almost all of those zeroed features were the noisy Customer ID/Name dummies, Lasso essentially performed automatic feature selection and rediscovered that `Loan Amount Request (USD)` (and a handful of other real features) is what actually matters, which is a nice practical demonstration of why Lasso is useful when you suspect a lot of your features are irrelevant or, like here, accidentally included by mistake.

9 Observations and Conclusion

All four models land in a fairly narrow test R2 range (0.515 – 0.520), so if you only looked at the test metrics table you would conclude they perform about the same. But that would be missing the actual story of this experiment: plain Linear Regression badly overfits due to `Customer ID` and `Name` accidentally being one-hot encoded as features, giving it a perfect (and meaningless) training R2 of 1.0. Ridge softens this a bit by shrinking coefficients, while Lasso and Elastic Net fix it properly by zeroing out almost all of the noisy identity-based features and keeping the genuinely useful ones like `Loan Amount Request (USD)`.

If I had to pick one model to actually deploy, **Lasso Regression** looks like the best choice here – best test RMSE (33260.77) and R2 (0.5191) among the fairer comparisons, smallest train-test gap (0.057, so it should generalize reliably to genuinely new applicants), and it also happens to double as a feature selector, cutting the model down to a small, interpretable set of real features instead of tens of thousands of memorized IDs. Elastic Net is a close second and would probably be the safer pick if the dataset changes slightly in the future, since it does not commit as hard to zeroing out features as Lasso does. In hindsight, the real lesson from this experiment is less about "which regularizer wins" and more about how easy it is to accidentally leak identity information into a model through careless feature typing, and how regularization can act as a safety net (though not a substitute) for careful feature selection.

10 References

[1] Scikit-learn Documentation – Linear Models. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html.

[2] Scikit-learn Documentation – Hyperparameter Optimization (GridSearchCV). [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html.

[3] Kaggle, "Predict Loan Amount Data." [Online]. Available: https://www.kaggle.com/.